

5. Evidências detalhadas por sprint

Recorte do relatório consolidado de evidências do MLX Pilot, mantendo somente a seção de detalhamento por sprint.

Sprint 1 - Fundação do projeto e baseline técnico

Campo	Descrição
Objetivo	Colocar a base do repositório, daemon, providers iniciais, catálogo e interface desktop em funcionamento.
Status consolidado	Concluída como fundação técnica do MVP.
Rastreabilidade Kanban/GitHub	Cards #1 a #12 identificados como Sprint 1.

Principais entregas evidenciadas

- Criação do workspace Cargo com crates core, providers e daemon.
- Definição de contratos de domínio para chat, provider e erros compartilhados.
- Provider MLX funcional para inferência local via CLI mlx-lm.
- Configuração por variáveis de ambiente para bind, modelos, provider e timeouts.
- Rotas HTTP principais para health, models, chat e catálogo remoto.
- Shell de UI desktop Tauri com áreas de Chat e Discover Models.
- Scripts de operação local para iniciar e parar o ambiente.

Evidências rastreáveis

- Kanban com cards #1 a #12 cobrindo workspace, core, MLX, daemon, catálogo, streaming, OpenClaw, rotas, UI e scripts.
- Cargo.toml na raiz define workspace Rust com crates core/providers/daemon.
- README_DEV documenta execução local, endpoints e build de daemon/desktop.
- Estrutura apps/desktop-ui e scripts/run-desktop.sh / stop-daemon.sh presentes no pacote.

Sprint 2 - UX principal de chat, layout e configurações

Campo	Descrição
Objetivo	Estruturar a experiência principal do usuário e ampliar o controle de providers, histórico e integrações OpenClaw/NanoBot.
Status consolidado	Concluída como sprint de maturação de experiência e operação.
Rastreabilidade Kanban/GitHub	Cards #13-#17 e #26-#30 identificados como Sprint 2.

Principais entregas evidenciadas

- Refino visual da interface desktop com layout profissional, topbar, ações de conversa e branding.
- Histórico com ações básicas de renomear e excluir conversas.
- Hub de configurações para OpenClaw, NanoBot, secrets e runtime.
- Aba OpenClaw com logs em tempo real e chat nativo.
- Runtime controls, observabilidade persistente e integração do provider Ollama no contexto OpenClaw.
- Diagnósticos, onboarding e detecção de install-state dos providers.
- Documentação principal reescrita com setup cross-platform.

Evidências rastreáveis

- Kanban com cards #13 a #17 e #26 a #30 atribuídos à Sprint 2.
- Arquivos UI: app.js, agent-control-plane.js, agent-channels.js, agent-skills.js, styles.css e index.html.
- Commits consultados incluem runtime controls/OpenClaw/Ollama provider e refinamentos de UI.
- README público e README_DEV descrevem arquitetura multi-provider, configuração e operação.

Sprint 3 - Multi-provider, observabilidade e resiliência inicial

Campo	Descrição
Objetivo	Consolidar execução com múltiplos backends, readiness operacional e primeiros recursos de observabilidade/fallback.
Status consolidado	Concluída como sprint de expansão do runtime e resiliência.
Rastreabilidade Kanban/GitHub	Cards #22, #23 e #33 identificados como Sprint 3.

Principais entregas evidenciadas

- Suporte funcional aos providers MLX, Ollama e llama.cpp.
- Roteamento por provider/modelo e documentação de runtime multi-provider.
- Fallback AIRLLM para cenários de memória restrita e modelos grandes.
- Console de observabilidade para depuração e auditoria do agente.
- Dashboard de observabilidade do daemon com logs, métricas e tracing.
- Ajustes de readiness, defaults e compatibilidade do daemon.

Evidências rastreáveis

- Kanban com cards #22, #23 e #33 vinculados à Sprint 3.
- README registra variáveis APP_MLX_AIRLLM_ENABLED, APP_MLX_AIRLLM_THRESHOLD_PERCENT, APP_MLX_AIRLLM_PYTHON_COMMAND, APP_MLX_AIRLLM_RUNNER e APP_MLX_AIRLLM_BACKEND.
- scripts/mlx_airllm_bridge.py existe no pacote e é referenciado no tauri.conf.json.
- Código de UI contém toggle AIRLLM e exibição de métrica airllm_used.

Sprint 4 - Agent core, tools, skills e hardening

Campo	Descrição
Objetivo	Construir o núcleo agentic local com execução multi-etapa, ferramentas sandboxadas, skills e políticas de segurança.
Status consolidado	Concluída como sprint central de agentic runtime e segurança.
Rastreabilidade Kanban/GitHub	Cards #18-#21, #24, #31 e #34 identificados como Sprint 4.

Principais entregas evidenciadas

- Scaffold e implementação das crates agent-core, agent-tools e agent-skills.
- AgentLoop com tool-calling, provider integration, prompt builder e contexto otimizado.
- Sistema de tools sandboxado com validação de schema, checkpoints e controles de execução.
- Loader de SKILL.md com parsing de frontmatter, limites, compatibilidade e hash de integridade.
- Endpoint /agent/run integrado ao daemon com wiring de estado.
- PolicyEngine, ApprovalService, AuditLog e Secrets Vault local criptografado.
- Hardening de segurança para chat e runtime.

Evidências rastreáveis

- Kanban com cards #18 a #21, #24, #31 e #34 vinculados à Sprint 4.
- docs/agent_architecture.md descreve fluxo UI -> daemon -> agent runtime -> provider/tools -> audit.
- Commits consultados registram scaffold de agent crates, sandboxed tools, SKILL.md loader, endpoint /agent/run e security layer.
- Crates agent-core, agent-tools e agent-skills possuem código Rust e contratos próprios.

Sprint 5 - Canais, control plane e release gate

Campo	Descrição
Objetivo	Ampliar operação multi-canal, preparar controles de release e documentar evolução do produto para ciclos seguintes.
Status consolidado	Concluída como sprint de operação, validação e preparação de release.
Rastreabilidade Kanban/GitHub	Cards #25, #32 e #35-#38 identificados como Sprint 5.

Principais entregas evidenciadas

- Control plane para canais oficiais e runtime multi-canal.
- Transports de canais com famílias bridge_http_v1, webhook_http_v1, token_bot_v1, matrix_http_v1, irc_tcp_v1 e native_runtime_v1.
- Release gate com critérios objetivos de GO/NO-GO, carga curta, migração/rollback, segurança, recuperação de falha e dry-run.
- Smoke tests de canais e skills no desktop.
- Evolução planejada para OODA, n8n embedded, plugins WASM e empacotamento multiplataforma.
- Documentação de protocolo e erros canônicos de canais.

Evidências rastreáveis

- Kanban com cards #25, #32, #35, #36, #37 e #38 vinculados à Sprint 5.
- docs/channel-transports.md e docs/channel-bridge-protocol-v1.md documentam canais, contas, credenciais e erros canônicos.
- docs/release-gate-report.md apresenta decisão final GO e ausência de blockers.
- apps/desktop-ui/e2e/channels-smoke.test.js e skills-smoke.test.js presentes no pacote.

Sprint 6 - Conversa persistente, observabilidade e unificação

Campo	Descrição
Objetivo	Melhorar continuidade de uso e consolidar componentes em uma experiência integrada.
Status consolidado	Registrada como sprint existente por documentação e código, com recomendação de criar cards de rastreabilidade no Kanban.
Rastreabilidade Kanban/GitHub	Não foram encontrados cards específicos S6 no Kanban consultado; evidência vem de documentação enviada e implementação.

Principais entregas evidenciadas

- Sessões de chat do agente com persistência local e fluxo de continuidade.
- Console de observabilidade com feed de auditoria e filtros.
- Unificação de MLX Server e MLX Pilot em uma experiência de produto mais integrada.
- Refinos de partículas/cena visual e interações desktop-native.
- Isolamento do codebase do agente e limpeza de artefatos de pesquisa.

Evidências rastreáveis

- Documento de Sprints enviado descreve explicitamente a Sprint 6.
- Commit consultado: feat(agent): implement active agent chat sessions local storage and UI.
- Commit consultado: feat(observability): agent console UI with audit feed and filters.
- Código em agent-control-plane.js e agent-core/session*.rs evidencia persistência, histórico e observabilidade.

Sprint 7 - AIRLLM fallback, resiliência de inferência e gestão de modelos

Campo	Descrição
Objetivo	Aumentar robustez para modelos pesados e cenários de memória limitada.
Status consolidado	Registrada como sprint existente por documentação e implementação técnica.
Rastreabilidade Kanban/GitHub	Rastreabilidade parcial no Kanban (#22 e #13), reforçada por documentação e código.

Principais entregas evidenciadas

- Fallback orquestrado via AIRLLM para modelos de alta memória.
- Exposição de sinais de runtime, métricas e histórico de uso.
- Toggle por requisição na UI para ativar fallback.
- Plumbing de backend/device hints no runtime.
- Melhorias de timeout, diagnósticos em tempo real, recuperação de paths, OOM e safe-mode.
- Gestão de modelos instalados com UX de rename/delete e correções de IDs MLX prefixados.

Evidências rastreáveis

- Documento de Sprints enviado descreve explicitamente a Sprint 7.
- README técnico registra variáveis APP_MLX_AIRLLM_* para ativação e configuração do fallback.
- scripts/mlx_airllm_bridge.py implementa bridge do fallback.
- UI contém botão AIRLLM, leitura de configuração, toggle de estado e renderização de métrica airllm_used.
- Kanban possui card #22 para Fallback AIRLLM e #13 para gerenciamento de modelos instalados, embora parte da implementação apareça em ciclo posterior.